МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта
Кафедра проблем управления

ПРАКТИЧЕСКАЯ РАБОТА №1

по дисциплине **Методы искусственного интеллекта**

Тема практической работы: «Применение метода градиентного спуска для аппроксимации зашумленных измерений»

Студенты группы: КРБО-03-23 Грачев А.В. _____

Зенина А.А. _____

Преподаватель: доцент Золотарёв А.А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

1. Цель работы

Изучение особенностей программной реализации метода градиентного спуска в задаче аппроксимации зашумленных измерений, представленных в виде набора точек, прямой линией.

2. Задача работы

Реализовать программно метод градиентного спуска с функцией ошибки - вертикальное рассогласование.

Выполнить дополнительное задание: реализовать градиентный спуск с модифицированным функционалом ошибки - ортогональное уклонение.

3. Теоретические сведения

В процессе своего функционирования робот непрерывно производит измерения разнородной сигнальной информации. Одним из эффективных подходов к обработке и анализу сигнальных данных является метод градиентного спуска. Этот алгоритм оптимизации широко используется в различных областях науки и техники для решения задач оптимизации и идентификации.

В контексте аппроксимации показаний датчиков метод градиентного спуска помогает оптимизировать параметры геометрической модели – прямой линии для описания выбранного участка графика.

Для случая аппроксимации набора значений линией эталонная модель прямой выбирается в формате:

$y = kx + b$,

где k – наклон прямой линии, а b – вертикальное смещение.

Рассогласование между некоторой прямой и точкой графика вычисляется по формуле:

$e_i(k, b) = y - y_i = kx_i + b - y_i$.

Функционал ошибки (среднеквадратичная ошибка) задаётся как:

$$E(k,b) = \frac{1}{2N}\sum_{i=1}^{N} e_i(k,b)^2$$

Частные производные функции ошибок по искомым параметрам:

$$\frac{\partial E}{\partial k} = \frac{1}{N}\sum_{i=1}^{N} x_i(kx_i + b - y_i),$$

$$\frac{\partial E}{\partial b} = \frac{1}{N}\sum_{i=1}^{N} (kx_i + b - y_i).$$

Тогда решение задачи аппроксимации сводится к итеративному выполнению следующих вычислений:

$$k_{t+1} = k_t + \eta_1 \frac{\partial E}{\partial k},$$

$$b_{t+1} = b_t + \eta_2 \frac{\partial E}{\partial b},$$

где η_1, η_2 скорости градиентного спуска по осям соответствующих параметров.

Лабораторная установка: среда разработки PyCharm Community Edition с использованием библиотеки pygame для визуализации.

4. Расчетно-графическая часть

Исходные данные:

Набор точек, имитирующих измерения:

pts = [[200, 200], [250, 260], [300, 300], [350, 360], [400, 420], [450, 490]]

Начальные параметры прямой: k = 0.7, b = 100.

Скорость обучения (эмпирически подобранная): $\eta = 0.1$ (для основного задания).

4.1. Результаты выполнения основного задания

Реализован алгоритм градиентного спуска для функционала MSE. Коэффициенты скорости обучения подобраны эмпирически: $\eta_1 = 0.00001$

(для k), $\eta_2 = 0.001$ (для b). Ниже приведена таблица с описанием ключевых функций программы.

Таблица 1.1 – Отчетная таблица

№	Наименование класса/функции	Комментарий
1	drawText(screen, s, x, y)	Отрисовка текста на экране Pygame
2	drawLine(screen)	Рисование текущей прямой (k, b) на экране
3	calcMSE()	Вычисление среднеквадратической ошибки для текущих k, b
4	gradDescent(eta)	Один шаг градиентного спуска, обновление k и b
5	Основной цикл while True	Анимация: 10000 шагов спуска, отрисовка, обновление экрана

На Рисунке 1 представлен результат аппроксимации после выполнения градиентного спуска. Зелёная линия отображает итоговую прямую, полученную в результате оптимизации параметров k и b.

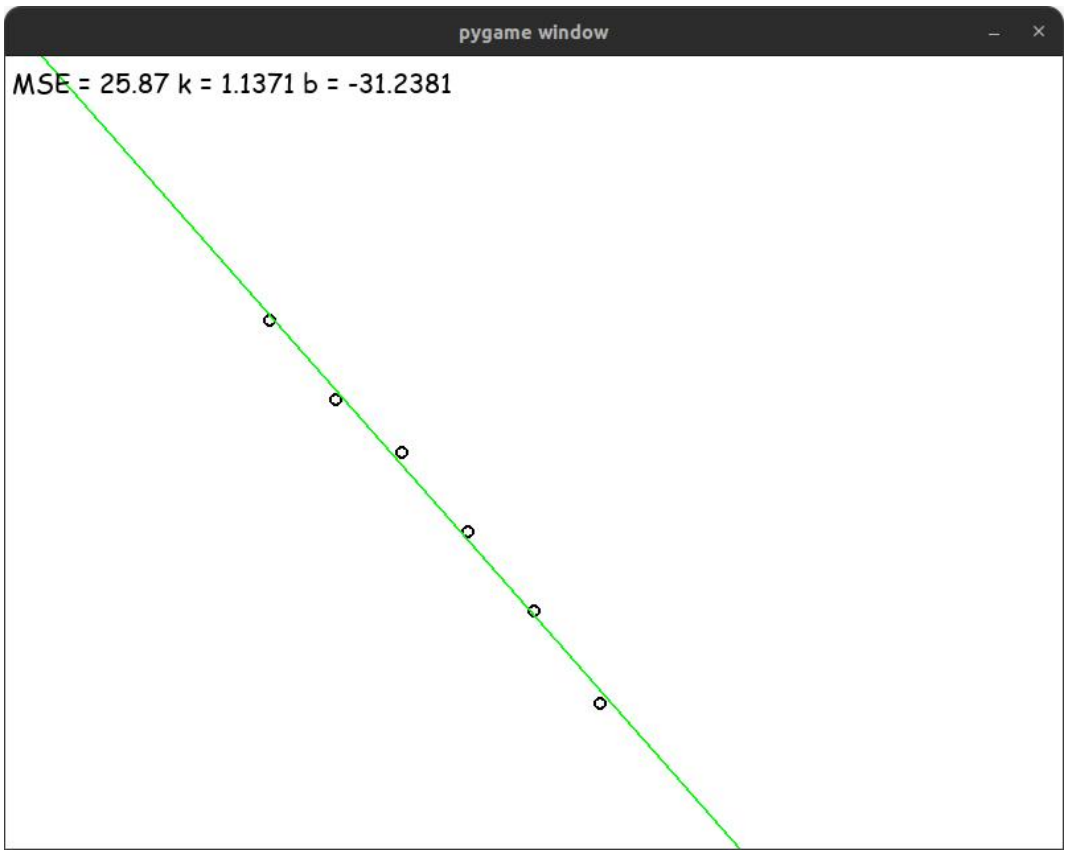

Рисунок 1 – Результат аппроксимации (основное задание)

Как видно из Рисунка 1, прямая линия достаточно точно аппроксимирует заданный набор точек. Это подтверждается низким значением среднеквадратичной ошибки, которое выводится на экран в процессе анимации.

4.2. Результаты выполнения дополнительного задания

В дополнительном задании функционал ошибки был модифицирован: вместо вертикальных рассогласований использовались ортогональные уклонения точек от искомой линии. Это означает, что расстояние от точки до прямой измеряется по перпендикуляру, что является более точной метрикой, но и более сложной в вычислительном плане.

На Рисунке 2 представлен результат работы модифицированного алгоритма.

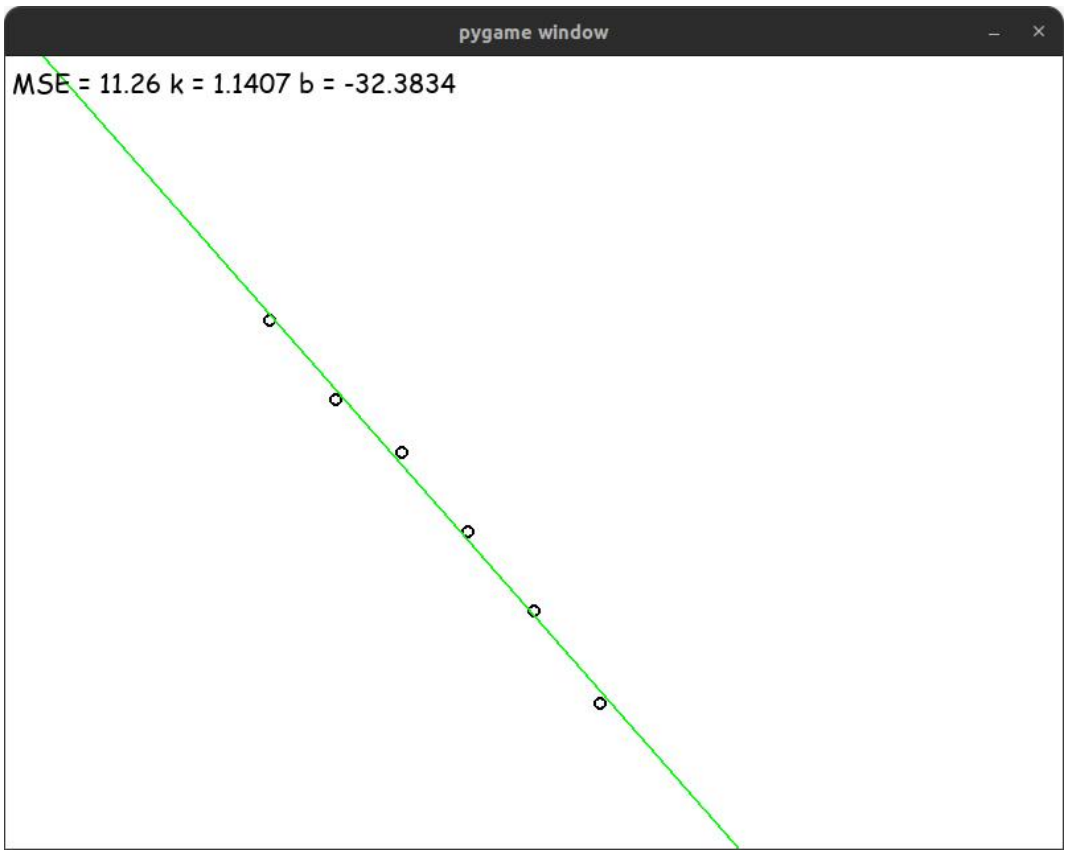

Рисунок 2 – Результат аппроксимации (дополнительное задание, ортогональные уклонения)

4.3. Анализ результатов

Сравнивая результаты основного и дополнительного заданий, можно сделать следующие выводы:

- Визуально обе прямые (полученные по MSE и по ортогональным уклонениям) одинаковые и и хорошо описывают облако точек, расхождения между ними начинаются в первом знаке после запятой.
- Использование ортогональных уклонений теоретически более корректно, так как учитывает ошибки по обеим координатам, а не только по y.

Таким образом, оба подхода успешно решают задачу аппроксимации, но модифицированный метод может быть предпочтительнее в случаях, когда данные зашумлены по обеим осям.

5. Выводы по работе

В ходе выполнения практической работы была успешно реализована программная модель аппроксимации набора точек прямой линией с использованием метода градиентного спуска.

Были изучены и реализованы два варианта функционала ошибки: на основе вертикальных рассогласований (стандартный MSE) и на основе ортогональных уклонений. Экспериментально подтверждена работоспособность обоих алгоритмов.

Цель работы, заключающаяся в изучении особенностей программной реализации метода градиентного спуска для аппроксимации зашумленных измерений, достигнута.

ПРИЛОЖЕНИЕ А

Листинг программы основного задания.

```
import sys, pygame

pygame.font.init()
font = pygame.font.SysFont('Comic Sans MS', 20)
def drawText(screen, s, x, y):
    screen.blit(font.render(s, True, (0,0,0)), (x,y))

sz = (800, 600)

pts=[
    [200, 200],
    [250, 260],
    [300, 300],
    [350, 360],
    [400, 420],
    [450, 490]
]

k=0.7
b=100
def drawLine(screen):
    p1=[0, k*0+b]
    p2=[800, k*800+b]
    pygame.draw.line(screen, (0,255,0), p1, p2, 2)

def calcMSE():
    res=0
    for p in pts:
        dy=p[1]-(k*p[0]+b)
        res+=dy*dy
    return res/(2*len(pts))

def gradDescent(eta):
    global k, b
    dEdk, dEdb = 0, 0
    for p in pts:
        dEdk += p[0]*(k*p[0] + b - p[1])
        dEdb += (k * p[0] + b - p[1])
    k-=eta*dEdk/len(pts)/10000
    b-=eta*dEdb/len(pts)/100

screen = pygame.display.set_mode(sz)
timer = pygame.time.Clock()
while True:
    for ev in pygame.event.get():
        if ev.type==pygame.QUIT:
            sys.exit(0)
    for i in range(10000):
        gradDescent(0.1) # выполнение шага градиентного спуска
    screen.fill((255, 255, 255))
    for p in pts:
        pygame.draw.circle(screen, (0,0,0), p, 5, 2)
    drawLine(screen)
    mse=calcMSE()
    drawText(screen, f"MSE = {mse:.2f} k = {k:.4f} b = {b:.4f}", 5, 5)
    pygame.display.flip()
    timer.tick(20)
```

ПРИЛОЖЕНИЕ Б

Листинг программы для выполнения дополнительного задания.

```
import sys, pygame

pygame.font.init()
font = pygame.font.SysFont('Comic Sans MS', 20)
def drawText(screen, s, x, y):
    screen.blit(font.render(s, True, (0,0,0)), (x,y))

sz = (800, 600)

pts=[
    [200, 200],
    [250, 260],
    [300, 300],
    [350, 360],
    [400, 420],
    [450, 490]
]

k=0.7
b=100
def drawLine(screen):
    p1=[0, k*0+b]
    p2=[800, k*800+b]
    pygame.draw.line(screen, (0,255,0), p1, p2, 2)

def calcMSE():
    res=0
    k2p1 = k*k + 1
    for p in pts:
        d = k * p[0] + b - p[1]
        res += (d * d) / k2p1
    return res/(2*len(pts))

def gradDescent(eta):
    global k, b
    dEdk, dEdb = 0, 0
    k2p1 = k*k + 1
    for p in pts:
        x = p[0]
        y = p[1]
        d = k * x + b - y
        dEdk += (d * x) / k2p1 - (k * d * d) / (k2p1 * k2p1)
        dEdb += d / k2p1
    k-=eta*dEdk/len(pts)/10000
    b-=eta*dEdb/len(pts)/100

screen = pygame.display.set_mode(sz)
timer = pygame.time.Clock()
while True:
    for ev in pygame.event.get():
        if ev.type==pygame.QUIT:
            sys.exit(0)
    for i in range(1000):
        gradDescent(0.1) # выполнение шага градиентного спуска
    screen.fill((255, 255, 255))
```

```
for p in pts:
    pygame.draw.circle(screen, (0,0,0), p, 5, 2)
drawLine(screen)
mse=calcMSE()
drawText(screen, f"MSE = {mse:.2f} k = {k:.4f} b = {b:.4f}", 5, 5)
pygame.display.flip()
timer.tick(60)
```